CHAPITRE 3 : MODÉLISATION DE L'APPLICATION EMBARQUÉE SUIVANT LES DIAGRAMMES UML

Contenu

1. Introduction à la modélisation pour systèmes embarqués
2. Diagramme de cas d'utilisation
3. Diagramme de classes
4. Diagramme d'états (State Machine)
5. Diagramme de séquence
6. Diagramme d'activité
7. Diagramme de déploiement
8. Diagramme de communication
9. Diagramme de structure composite
10. Outils de modélisation
11. Exercices d'application

3.1 INTRODUCTION À LA MODÉLISATION POUR SYSTÈMES EMBARQUÉS

3.1.1 Pourquoi modéliser un système embarqué ?

La modélisation est une étape cruciale dans le développement de systèmes embarqués pour plusieurs raisons :

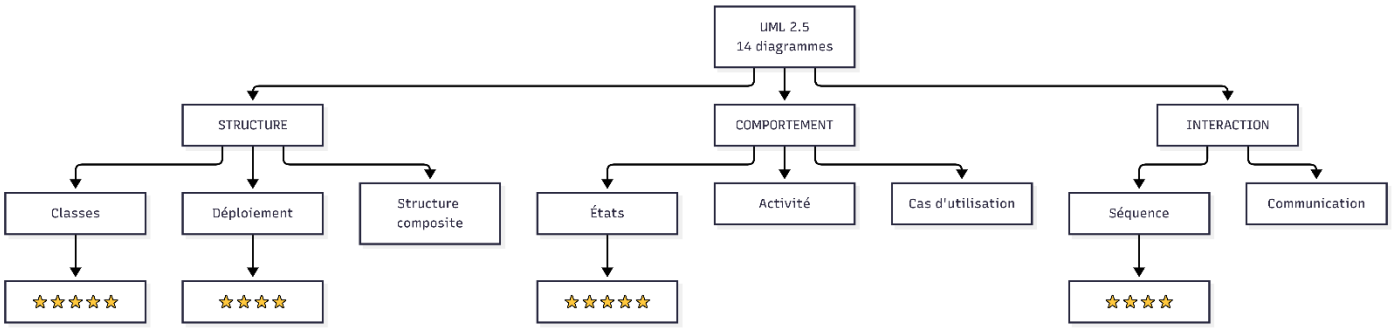
- **COMPRÉHENSION** : Visualiser le système avant son implémentation
- **COMMUNICATION** : Langage commun entre les parties prenantes
- **VALIDATION** : Vérifier la cohérence et détecter les erreurs
- **DOCUMENTATION** : Tracer les décisions de conception
- **GÉNÉRATION DE CODE** : Certains outils permettent de générer automatiquement du code à partir des modèles
- **RÉUTILISATION** : Les modèles peuvent être adaptés pour d'autres projets

3.1.2 Spécificités des systèmes embarqués pour la modélisation

Caractéristique	Impact sur la modélisation
Temps réel	Nécessité de modéliser les contraintes temporelles
Concurrence	Plusieurs tâches s'exécutent en parallèle
Interactions matérielles	Interface avec capteurs/actionneurs à modéliser
États finis	Comportement souvent décrit par machines d'états
Ressources limitées	Contraintes mémoire/temps à intégrer

Sûreté de fonctionnement	Modélisation des modes dégradés et erreurs

3.1.3 Vue d'ensemble des diagrammes UML pour l'embarqué



3.2 DIAGRAMME DE CAS D'UTILISATION

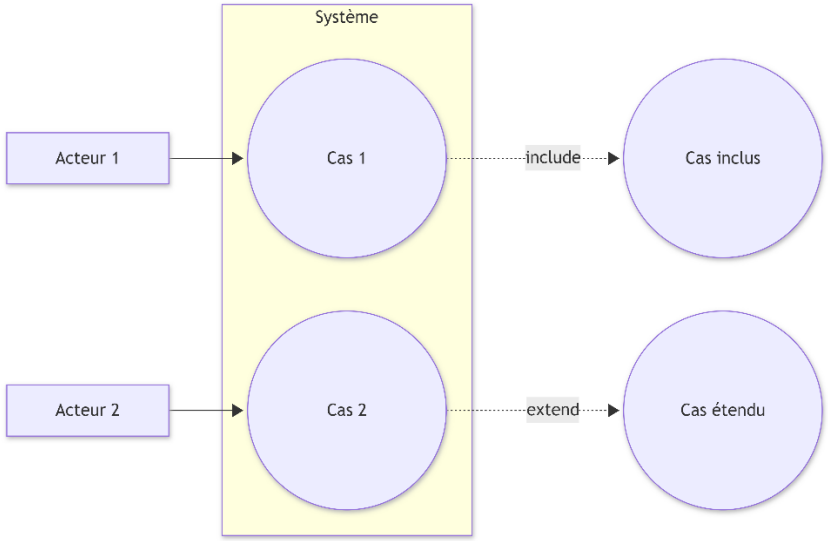
3.2.1 Définition et objectifs

Le diagramme de cas d'utilisation décrit les fonctionnalités du système du point de vue de l'utilisateur.

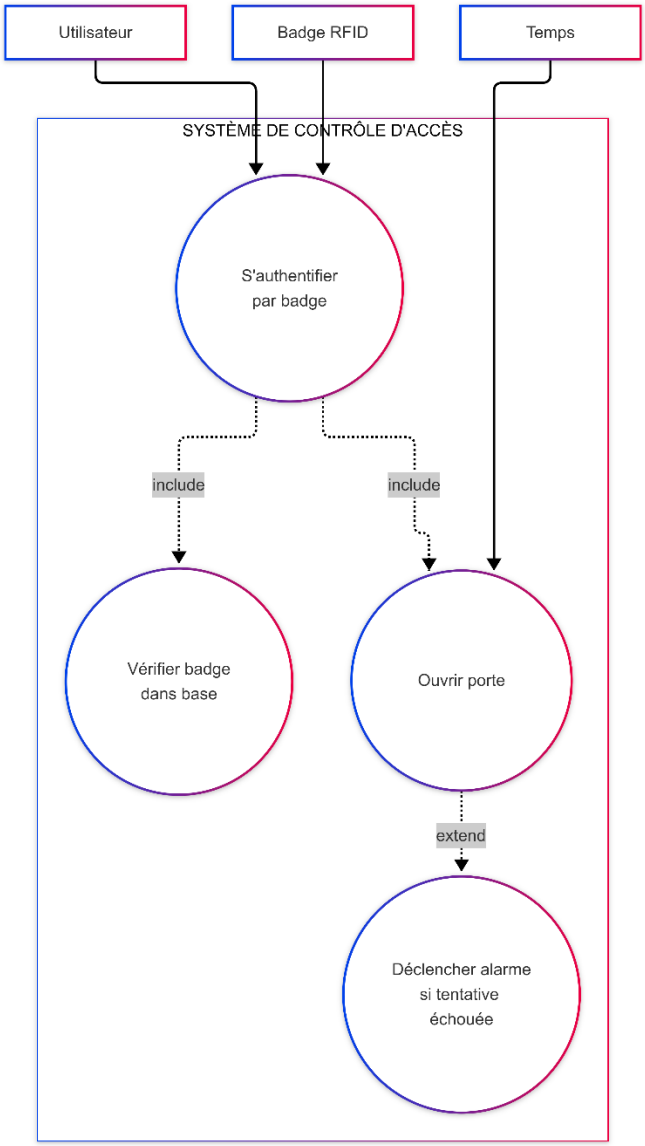
Objectifs :

- Capturer les exigences fonctionnelles
- Délimiter le périmètre du système
- Identifier les acteurs et leurs interactions
- Servir de base pour les tests d'acceptation

3.2.2 Éléments du diagramme de cas d'utilisation



3.2.3 Exemple : Système de contrôle d'accès RFID



3.2.4 Acteurs typiques dans les systèmes embarqués

Type d'acteur	Exemples	Représentation
Humain	Utilisateur, opérateur, administrateur	Bonhomme stylisé
Système externe	Base de données, cloud, autre système	Rectangle avec « système »
Capteur	Température, pression, mouvement	Stéréotype « capteur »
Actionneur	Moteur, relais, afficheur	Stéréotype « actionneur »
Temps	Horloge temps réel, timer	Stéréotype « time »

3.3 DIAGRAMME DE CLASSES

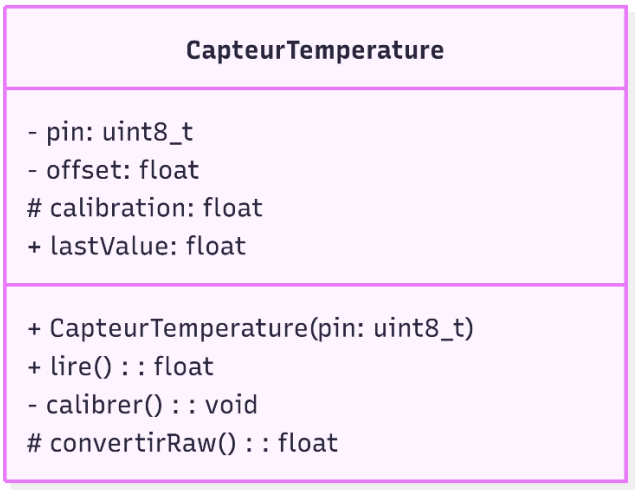
3.3.1 Définition et objectifs

Le diagramme de classes décrit la structure statique du système : les entités logicielles, leurs attributs, leurs méthodes et leurs relations.

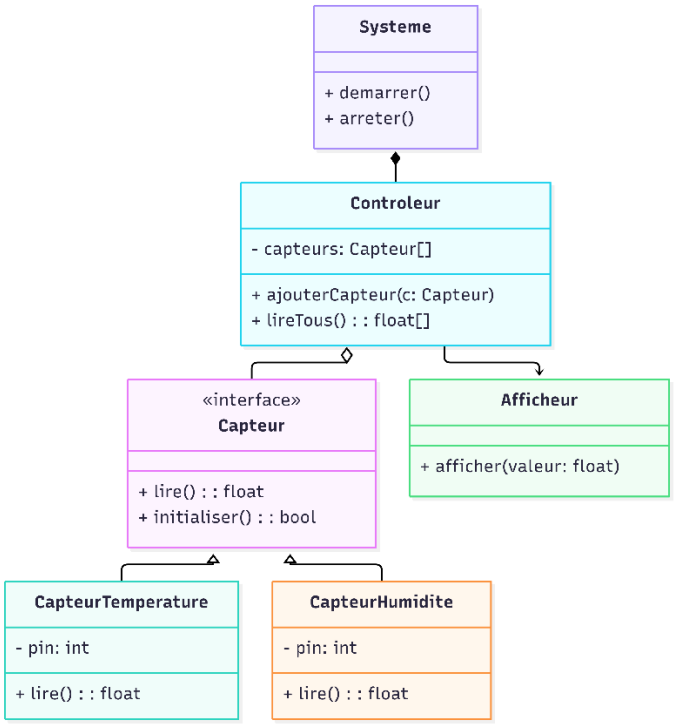
Objectifs :

- Modéliser la structure du logiciel
- Définir les interfaces entre composants
- Préparer l'implémentation orientée objet
- Documenter l'architecture logicielle

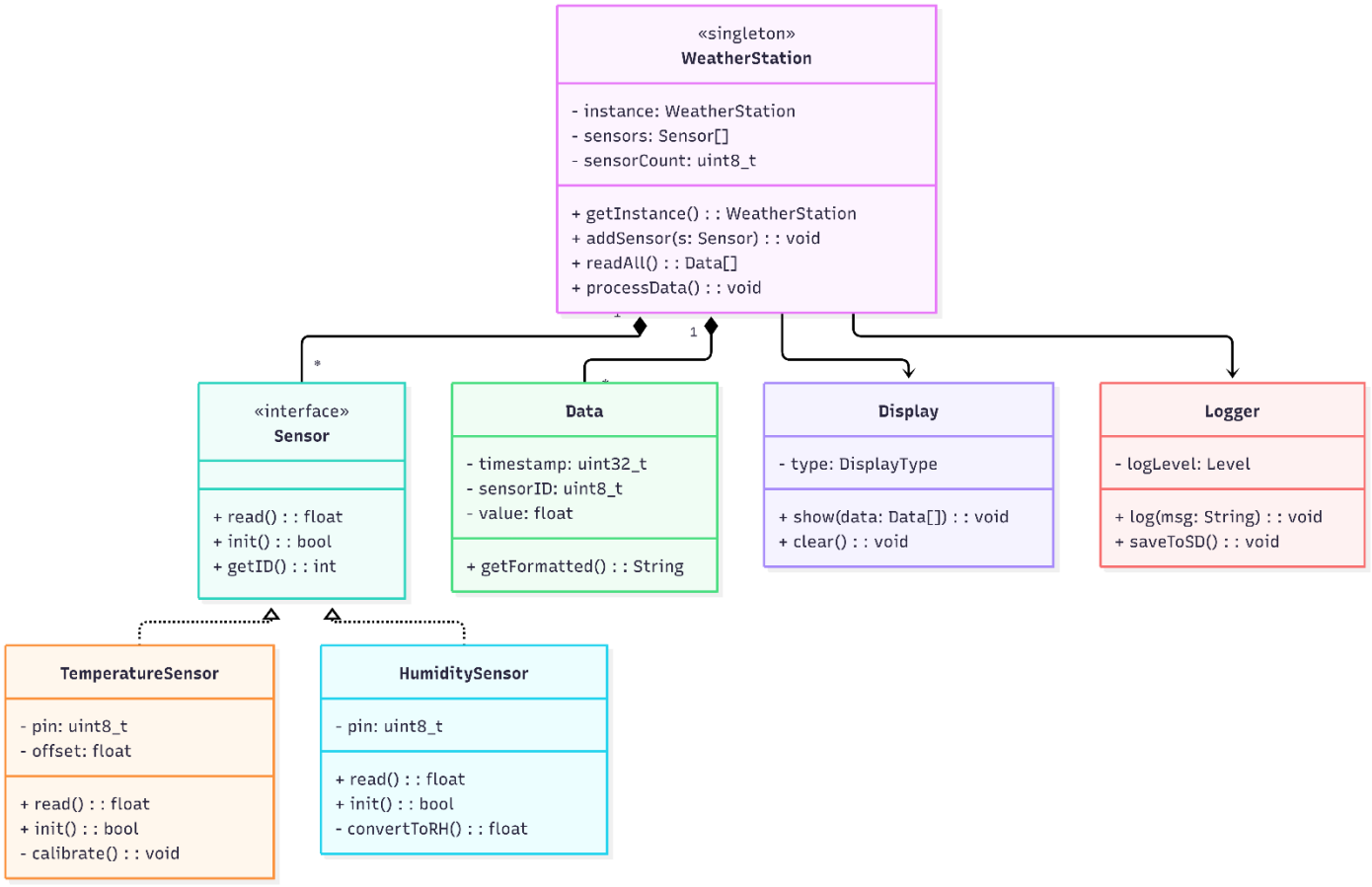
3.3.2 Représentation d'une classe UML



3.3.3 Relations entre classes



3.3.4 Exemple complet : Système de station météo



3.3.5 Implémentation C++ pour embarqué

```cpp
// Interface Sensor
class Sensor {
public:
    virtual float read() = 0;
    virtual bool init() = 0;
    virtual uint8_t getID() = 0;
    virtual ~Sensor() {}
};

// Capteur de température concret
class TemperatureSensor : public Sensor {
private:
    uint8_t pin;
    float offset;
    void calibrate() { /* calibration interne */ }

public:
    TemperatureSensor(uint8_t p) : pin(p), offset(0.0) {}
```

```
    float read() override {
        return 22.5; // Valeur simulée
    }

    bool init() override {
        pinMode(pin, INPUT);
        calibrate();
        return true;
    }

    uint8_t getID() override { return 0x01; }
};

// Singleton WeatherStation
class WeatherStation {
private:
    static WeatherStation* instance;
    Sensor* sensors[10];
    uint8_t sensorCount;

    WeatherStation() : sensorCount(0) {}

public:
    static WeatherStation* getInstance() {
        if (!instance) {
            instance = new WeatherStation();
        }
        return instance;
    }

    void addSensor(Sensor* s) {
        if (sensorCount < 10) {
            sensors[sensorCount++] = s;
        }
    }
};
```

3.4 DIAGRAMME D'ÉTATS (STATE MACHINE)

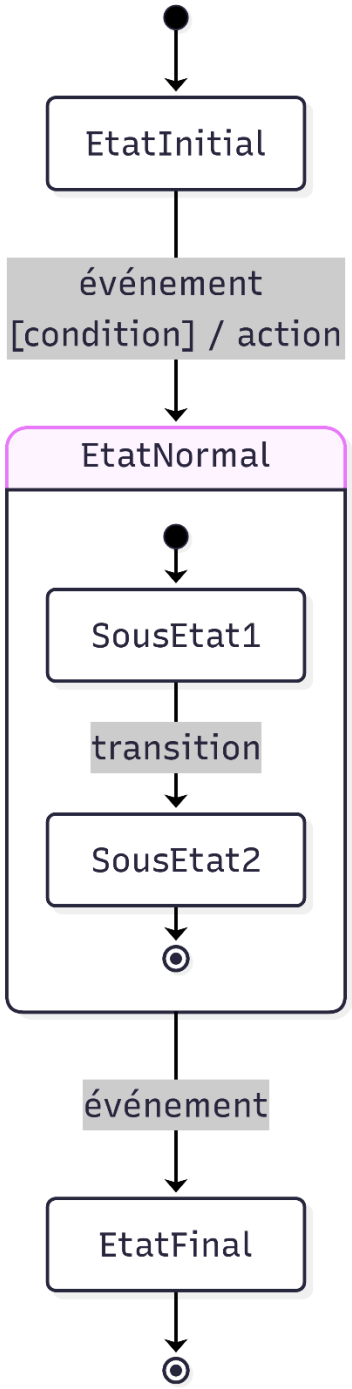
3.4.1 Définition et objectifs

Le diagramme d'états modélise le comportement dynamique d'un système en réponse à des événements. C'est le diagramme le plus important pour les systèmes embarqués réactifs.

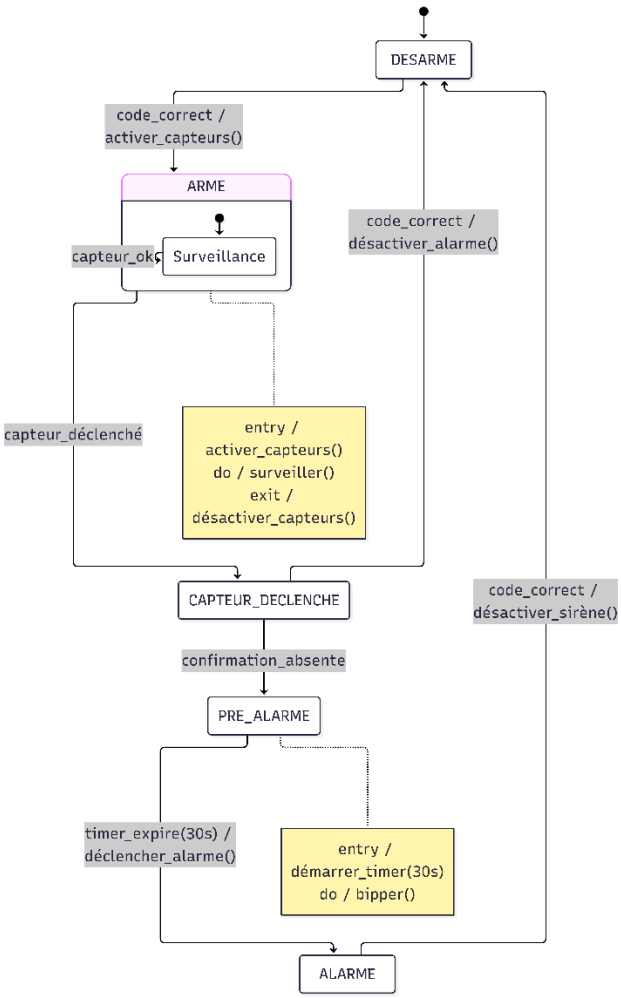
Objectifs :

- Modéliser les modes de fonctionnement
- Décrire les transitions entre états
- Spécifier les actions déclenchées par les événements
- Gérer les comportements temps réel

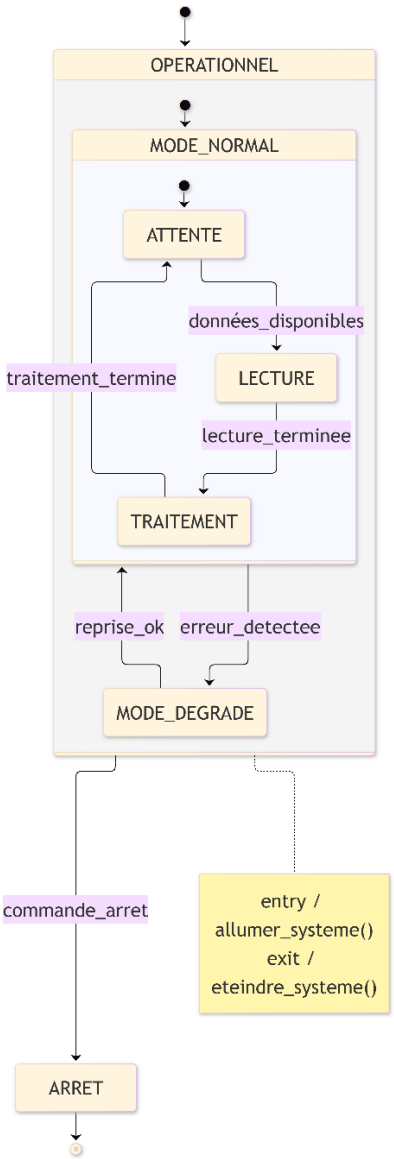
3.4.2 Éléments d'un diagramme d'états



3.4.3 Exemple : Système d'alarme



3.4.4 Machine d'états hiérarchiques



3.4.5 Implémentation en C d'une machine d'états

```c
// Définition des états
typedef enum {
    STATE_DESARME,
    STATE_ARME,
    STATE_CAPTEUR_DECLENCHE,
    STATE_PRE_ALARME,
    STATE_ALARME
} SystemState;

// Définition des événements
typedef enum {
    EVENT_CODE_CORRECT,
    EVENT_CODE_INCORRECT,
    EVENT_CAPTEUR_DECLENCHE,
    EVENT_CONFIRMATION,
```

```
    EVENT_TIMER_EXPIRE,
    EVENT_MAX
} SystemEvent;

// Structure de la machine d'états
typedef struct {
    SystemState currentState;
    uint32_t timerValue;
    uint8_t attemptCount;
} StateMachine;

// Table de transition (état, événement) -> nouvel état
SystemState transitionTable[STATE_ALARME+1][EVENT_MAX] = {
    // STATE_DESARME
    { STATE_ARME, STATE_DESARME, STATE_DESARME, STATE_DESARME,
STATE_DESARME },
    // STATE_ARME
    { STATE_DESARME, STATE_ARME, STATE_CAPTEUR_DECLENCHE, STATE_ARME,
STATE_ARME },
    // STATE_CAPTEUR_DECLENCHE
    { STATE_DESARME, STATE_CAPTEUR_DECLENCHE, STATE_PRE_ALARME,
STATE_DESARME, STATE_ARME },
    // STATE_PRE_ALARME
    { STATE_DESARME, STATE_PRE_ALARME, STATE_PRE_ALARME, STATE_DESARME,
STATE_ALARME },
    // STATE_ALARME
    { STATE_DESARME, STATE_ALARME, STATE_ALARME, STATE_DESARME,
STATE_ALARME }
};

// Fonctions d'actions
void entry_desarme() { /* ... */ }
void entry_arme() { activer_capteurs(); }
void entry_capteur_declenche() { allumer_LED(); }
void entry_pre_alarme() { demarrer_timer(30); }
void entry_alarme() { activer_sirene(); }

// Gestionnaire d'événements
void handleEvent(StateMachine* sm, SystemEvent event) {
    SystemState newState = transitionTable[sm->currentState][event];

    if (newState != sm->currentState) {
        // Exit actions
        switch(sm->currentState) {
            case STATE_ARME: desactiver_capteurs(); break;
            case STATE_CAPTEUR_DECLENCHE: eteindre_LED(); break;
            case STATE_PRE_ALARME: arreter_timer(); break;
```

```
            case STATE_ALARME: eteindre_sirene(); break;
            default: break;
        }

        sm->currentState = newState;

        // Entry actions
        switch(newState) {
            case STATE_DESARME: entry_desarme(); break;
            case STATE_ARME: entry_arme(); break;
            case STATE_CAPTEUR_DECLENCHE: entry_capteur_declenche();
break;
            case STATE_PRE_ALARME: entry_pre_alarme(); break;
            case STATE_ALARME: entry_alarme(); break;
            default: break;
        }
    }
}
```

3.5 DIAGRAMME DE SÉQUENCE

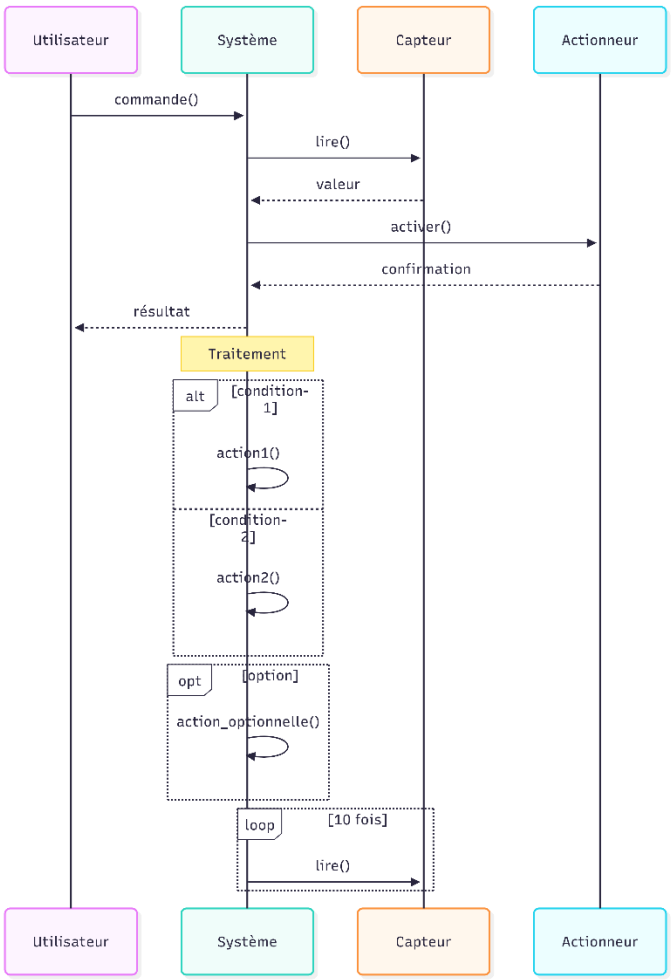
3.5.1 Définition et objectifs

Le diagramme de séquence montre les interactions entre objets dans le temps.

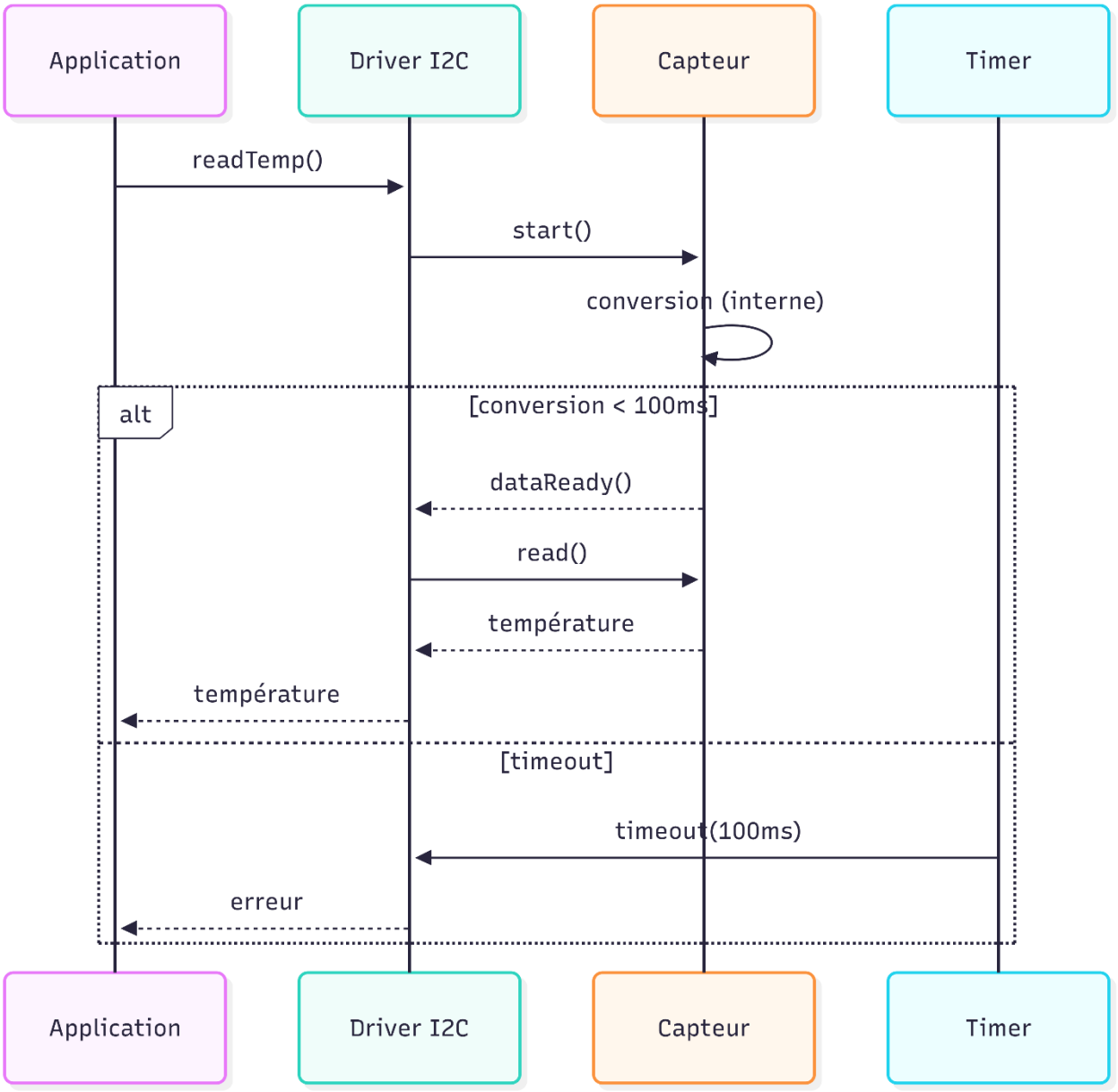
Objectifs :

- Visualiser le flux de messages entre composants
- Modéliser des scénarios d'utilisation spécifiques
- Détail les échanges pour un cas d'utilisation
- Valider la chronologie des opérations

3.5.2 Éléments d'un diagramme de séquence



3.5.3 Exemple : Lecture de capteur avec timeout



3.5.4 Implémentation du scénario en C

```
c
float readTemperatureWithTimeout(uint8_t sensorAddr, uint32_t timeoutMs)
{
    uint32_t startTime = millis();

    // Démarrer la conversion
    i2c_start_conversion(sensorAddr);

    // Attendre avec timeout
    while (millis() - startTime < timeoutMs) {
        if (i2c_is_conversion_complete(sensorAddr)) {
            return i2c_read_temperature(sensorAddr);
        }
```

```
        delay(1);
    }

    return -273.15; // Valeur d'erreur
}
```

3.6 DIAGRAMME D'ACTIVITÉ

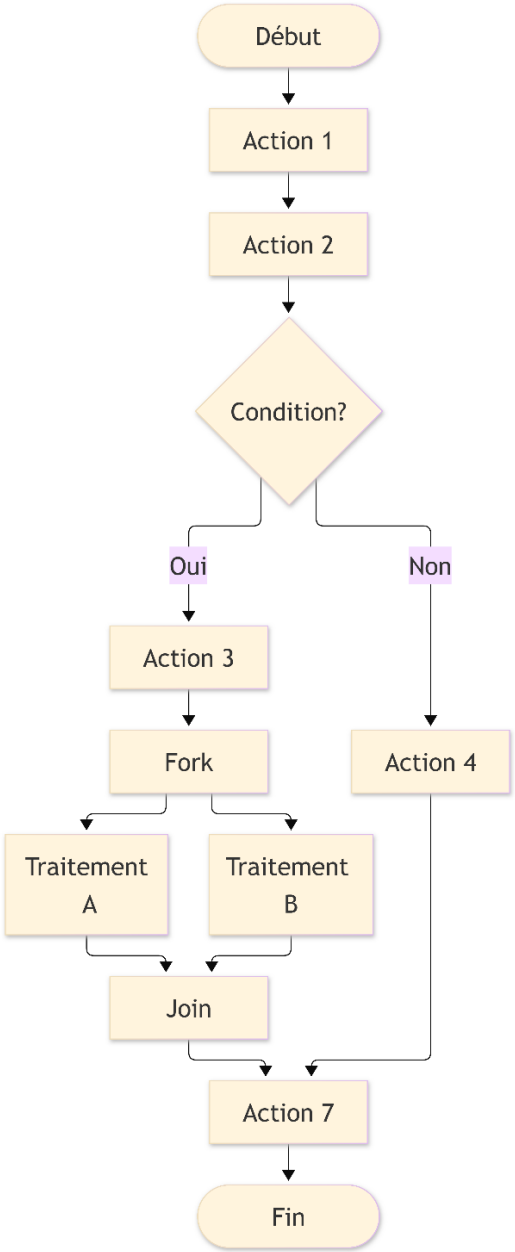
3.6.1 Définition et objectifs

Le diagramme d'activité modélise le flux de contrôle et de données dans un processus.

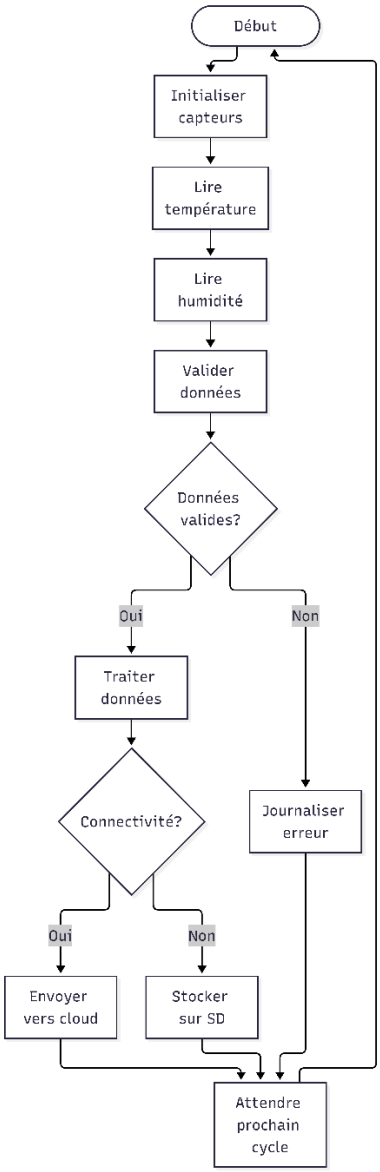
Objectifs :

- Décrire des algorithmes complexes
- Modéliser des traitements parallèles
- Visualiser des workflows
- Compléter les diagrammes d'états pour les aspects procéduraux

3.6.2 Éléments d'un diagramme d'activité



3.6.3 Exemple : Acquisition et traitement de données



3.7 DIAGRAMME DE DÉPLOIEMENT

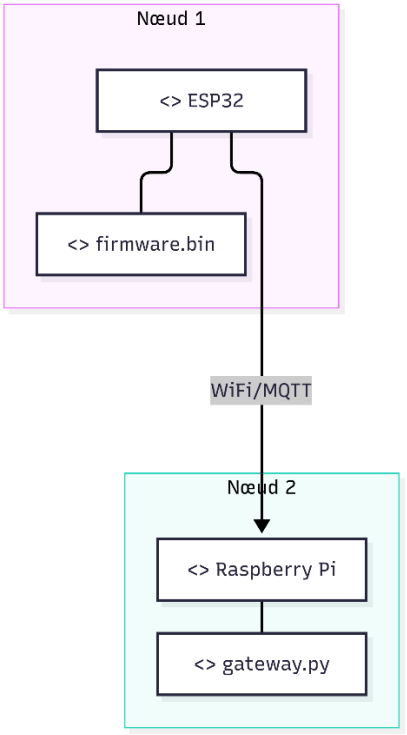
3.7.1 Définition et objectifs

Le diagramme de déploiement montre l'architecture physique du système : les nœuds matériels et la répartition des composants logiciels.

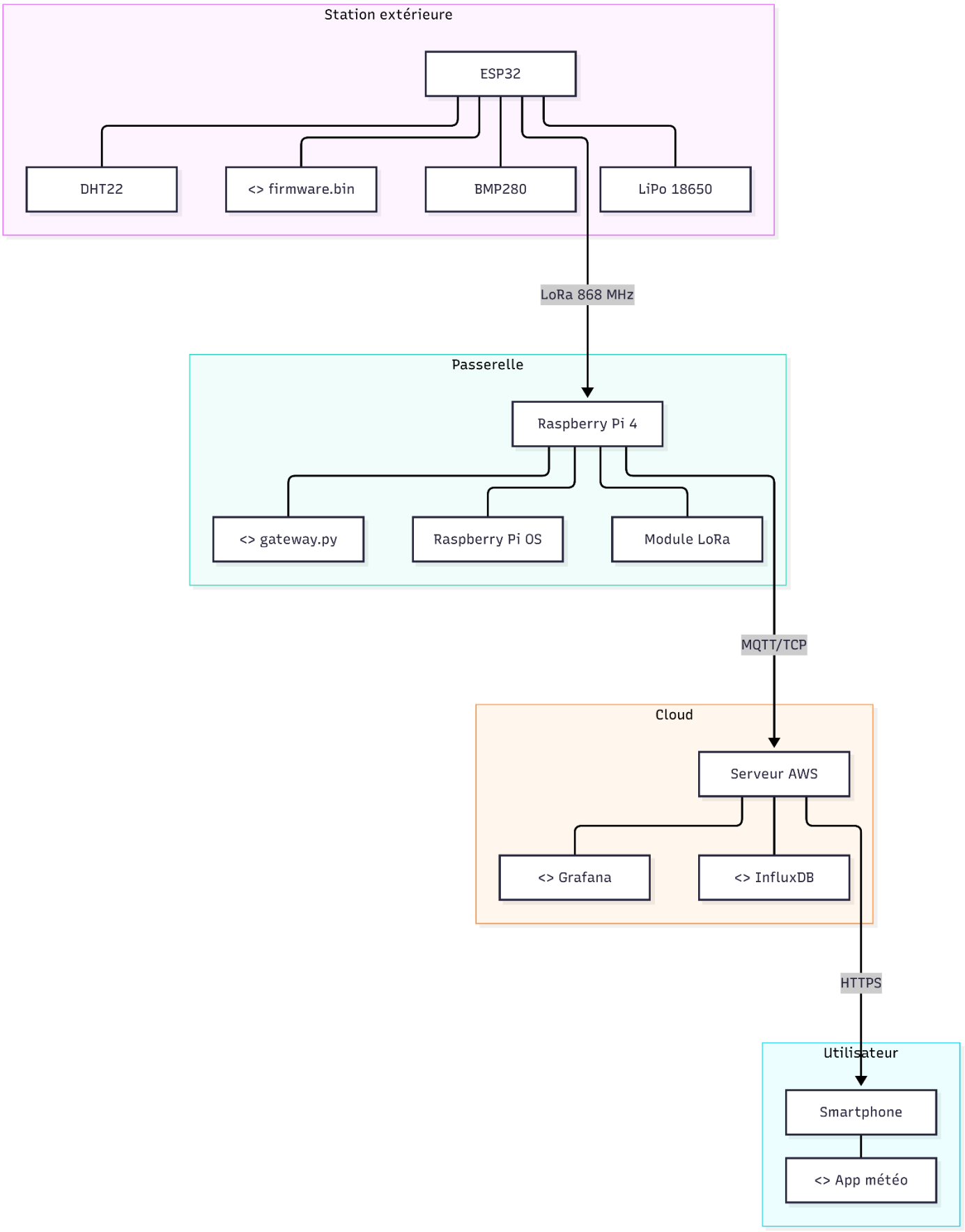
Objectifs :

- Visualiser l'architecture matérielle
- Montrer la répartition des composants logiciels
- Spécifier les interfaces de communication
- Planifier le déploiement physique

3.7.2 Éléments d'un diagramme de déploiement



3.7.3 Exemple : Station météo IoT


Station extérieure
ESP32
DHT22
<> firmware.bin
BMP280
LiPo 18650
LoRa 868 MHz
Passerelle
Raspberry Pi 4
<> gateway.py
Raspberry Pi OS
Module LoRa
MQTT/TCP
Cloud
Serveur AWS
<> Grafana
<> InfluxDB
HTTPS
Utilisateur
Smartphone
<> App météo

3.8 DIAGRAMME DE COMMUNICATION

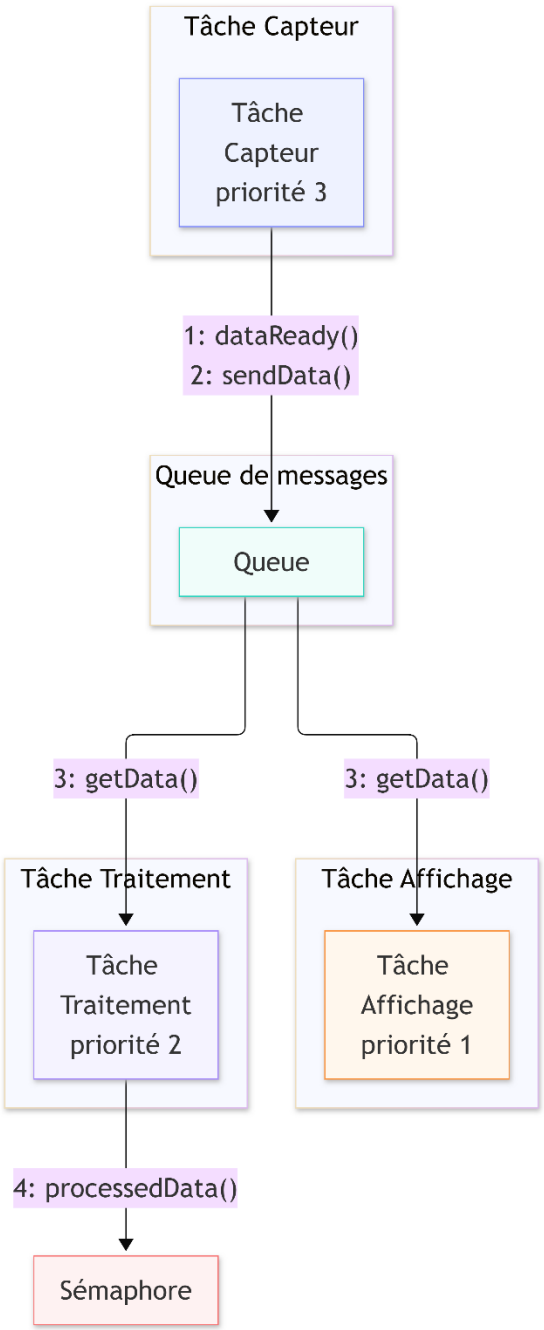
3.8.1 Définition et objectifs

Le diagramme de communication montre les interactions entre objets avec un focus sur l'organisation structurelle.

Objectifs :

- Montrer les liens entre objets
- Visualiser les échanges dans leur contexte structurel
- Compléter le diagramme de séquence pour l'aspect organisationnel

3.8.2 Exemple : Communication entre tâches RTOS



3.9 DIAGRAMME DE STRUCTURE COMPOSITE

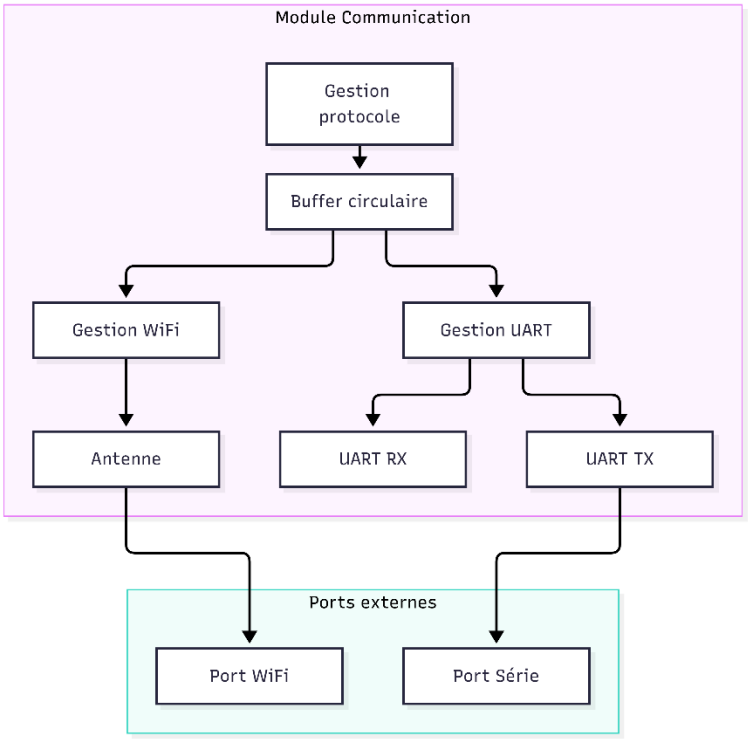
3.9.1 Définition et objectifs

Le diagramme de structure composite montre l'organisation interne d'un composant complexe.

Objectifs :

- Détailler l'architecture interne d'un composant
- Montrer les ports et interfaces
- Visualiser les connexions entre parties internes

3.9.2 Exemple : Module de communication



3.10 OUTILS DE MODÉLISATION POUR L'EMBARQUÉ

3.10.1 Outils UML généraux

Outil	Type	Avantages	Inconvénients
Enterprise Architect	Commercial	Complet, génération de code	Coût élevé
Visual Paradigm	Commercial	Très complet	Courbe d'apprentissage
Papyrus (Eclipse)	Open source	Gratuit, standard OMG	Complexe
Draw.io / **Diagrams.net**	Gratuit	Simple, en ligne	Pas de génération de code

Mermaid	Open source	Intégré Markdown, simple	Limité pour très grands projets

3.10.2 Outils spécialisés pour l'embarqué

Outil	Description	Génération de code
MATLAB/Simulink	Modélisation système, Stateflow	C, C++, HDL
SCADE	Pour systèmes critiques	C, Ada, certifié
Rhapsody (IBM)	UML pour embarqué	C, C++, Java
Yakindu Statechart Tools	Spécialisé machines d'états	C, C++, Java

---

3.11 EXERCICES DE CONSOLIDATION

Exercice 1 : Cas d'utilisation pour un système d'arrosage automatique (2 pts)

Un système d'arrosage automatique doit :

- Être programmable (jours/heures)
- Détecter la pluie pour annuler l'arrosage
- Surveiller l'humidité du sol
- Déclencher une alarme en cas de fuite

Questions :

1. Identifiez les acteurs du système
2. Dessinez le diagramme de cas d'utilisation avec Mermaid
3. Détaillez le cas "Programmer arrosage" avec les relations include/extend

---

Exercice 2 : Diagramme de classes pour un vélo électrique (3 pts)

Un vélo électrique possède :

- Une batterie (charge, tension, courant)
- Un moteur (puissance, vitesse)
- Un contrôleur qui gère l'assistance
- Un afficheur (vitesse, niveau batterie)
- Des capteurs (cadence de pédalage, vitesse)

Questions :

1. Identifiez les classes principales

2. Définissez leurs attributs et méthodes
3. Spécifiez les relations entre classes
4. Proposez un diagramme de classes UML avec Mermaid

Exercice 3 : Machine d'états pour un portail automatique (3 pts)

Un portail automatique fonctionne ainsi :

- Initialement fermé
- Appui télécommande → ouverture (10s)
- Si obstacle pendant ouverture → s'arrête puis recule
- Reste ouvert 30s puis fermeture automatique
- Appui pendant fermeture → rouvre
- Détection cellule photoélectrique pendant fermeture → rouvre

Questions :

1. Identifiez les états du système
2. Listez les événements
3. Dessinez le diagramme d'états avec Mermaid
4. Ajoutez les actions entry/do/exit

Exercice 4 : Diagramme de séquence pour une transaction RFID (3 pts)

Scénario : Un utilisateur présente son badge RFID pour ouvrir une porte.
Le système doit :

1. Lire l'UID du badge
2. Vérifier dans la base locale
3. Si autorisé, ouvrir la porte (2s) et logger l'accès
4. Si non autorisé, afficher "Accès refusé" et logger la tentative

Questions :

1. Identifiez les objets participants
2. Dessinez le diagramme de séquence avec Mermaid
3. Ajoutez un cas alternatif pour badge invalide

Exercice 5 : Diagramme de déploiement pour une maison intelligente (3 pts)

Une maison intelligente comprend :

- Des capteurs (température, luminosité) sur ESP32
- Des actionneurs (volets, lumières) sur ESP32
- Une passerelle centrale sur Raspberry Pi
- Un serveur cloud pour le stockage et l'analyse
- Une application mobile pour le contrôle

Questions :

1. Identifiez les nœuds matériels
2. Spécifiez les composants logiciels déployés
3. Indiquez les protocoles de communication
4. Dessinez le diagramme de déploiement avec Mermaid

Exercice 6 : Diagramme d'activité pour un système de sécurité (3 pts)

Un système de sécurité fonctionne ainsi :

- Surveillance continue des capteurs
- Si mouvement détecté : vérifier code alarme
- Si code correct : désactiver l'alarme
- Si code incorrect : déclencher sirène + notification
- Si pas de mouvement : continuer surveillance

Questions :

1. Identifiez les actions du système
2. Identifiez les décisions
3. Dessinez le diagramme d'activité avec Mermaid
4. Ajoutez un traitement parallèle pour la notification

Exercice 7 : Synthèse - Système de distributeur automatique (3 pts)

Concevez la modélisation complète d'un distributeur automatique de boissons :

Fonctionnalités :

- Affichage des boissons disponibles

- Insertion de pièces
- Sélection de boisson
- Distribution et rendu de monnaie
- Annulation possible
- Maintenance (rechargement)

Questions :

1. Diagramme de cas d'utilisation
2. Diagramme de classes
3. Diagramme d'états pour le cycle de distribution
4. Diagramme de séquence pour une transaction réussie
5. Diagramme de déploiement

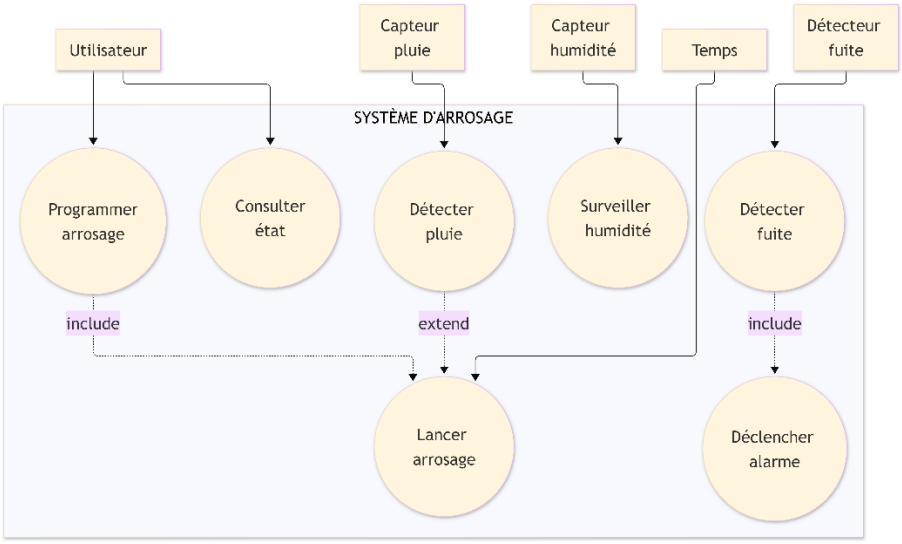
3.12 CORRIGÉS DES EXERCICES

Corrigé Exercice 1 : Cas d'utilisation arrosage automatique

1. Acteurs identifiés :

- Utilisateur (programmation, consultation)
- Capteur de pluie
- Capteur d'humidité
- Détecteur de fuite
- Temps (programmateur)

2. Diagramme de cas d'utilisation :



3. Détail du cas "Programmer arrosage" :

- Le cas "Programmer arrosage" inclut "Lancer arrosage" (obligatoire)
- Le cas "Détecter pluie" étend "Lancer arrosage" (optionnel : si pluie, annule)

Corrigé Exercice 2 : Diagramme de classes vélo électrique

1. Classes identifiées :

- Batterie
- Moteur
- Contrôleur
- Afficheur
- CapteurCadence

- CapteurVitesse

2. Diagramme de classes :

```
classDiagram
    class Batterie {
        - charge: float
        - tension: float
        - courant: float
        + getCharge(): float
        + getTension(): float
        + estVide(): bool
    }

    class Moteur {
        - puissance: float
        - vitesse: float
        + setPuissance(p: float)
        + getVitesse(): float
    }

    class Controleur {
        - niveauAssistance: int
        + calculerAssistance(cadence: float): float
        + commanderMoteur(p: float)
        + verifierBatterie(): bool
    }

    class Afficheur {
        + afficherVitesse(v: float)
        + afficherBatterie(niveau: float)
    }
```

```
class CapteurCadence {
    + lire(): float
}

class CapteurVitesse {
    + lire(): float
}

Controleur *-- Batterie
Controleur *-- Moteur
Controleur *-- CapteurCadence
Controleur *-- CapteurVitesse
Controleur --> Afficheur
```

